

Урок 20. Корутины. Flow. Билдеры и простые операторы.

28 августа 2020

В этом уроке рассмотрим способы создания Flow, простые операторы и создание своих операторов.

Билдеры

Основной способ создания Flow - это билдер flow, который мы уже рассматривали в прошлом уроке:

```
1 | val flow = flow {
2 |     emit("a")
3 |     emit("b")
4 |     emit("c")
5 | }
```

В билдер мы передаем блок кода, который будет запущен, когда получатель запустит этот Flow. Методом emit мы отправляем данные получателю.

Также существуют билдеры-обертки asFlow и flowOf, которые избавляют нас от написания очевидного и простого кода, а под капотом используют билдер flow.

Пример создания Flow из коллекции с помощью asFlow:

```
1 | val flow = listOf("a", "b", "c").asFlow()
```

Под капотом у asFlow все очень просто:

```
1 | public fun <T> Iterable<T>.asFlow(): Flow<T> = flow {
2 |     forEach { value ->
3 |         emit(value)
4 |     }
5 | }
```

Используется билдер flow. В нем идет перебор элементов коллекции и их отправка получателю методом emit.

Билдер flowOf может сделать то же самое еще лаконичнее. На вход он принимает vararg:

```
1 | val flow = flowOf("a", "b", "c")
```

А если у нас есть suspend функция, которая ничего не принимает на вход и возвращает результат:

```
1 | suspend fun getData(): Data
```

то можно использовать ссылку на нее, чтобы создать Flow:

```
1 | val flow = ::getData.asFlow()
```

Операторы

Основные операторы для Flow бывают двух типов: Intermediate и Terminal. Первые добавляют в Flow различные преобразования данных, но не запускают его. А вторые - запускают Flow и работают с результатом его работы.

Мы создадим свои операторы обоих типов, чтобы лучше понять разницу.

Intermediate

Примеры Intermediate операторов - map, filter, take, zip, combine, withIndex, scan, debounce, distinctUntilChanged, drop, sample. Все они знакомы нам по Kotlin коллекциям, RxJava и Java Stream, поэтому нет смысла рассматривать что делает каждый из них.

В качестве простого примера, как они используются с Flow, рассмотрим оператор map:

```
1 | val flow = flowOf(1,2,3).map { it * 10 }
```

Оператор map умножит на 10 значения 1,2,3, которые придут из Flow. В итоге получится новый Flow, который вернет 10,20,30. При этом оператор map сам по себе не запускает Flow, к которому мы его применяем. Он просто создает новый Flow-обертку поверх старого. Работать он начнет, когда мы вызовем collect.

Давайте на примере создания своего оператора разберемся, как работает такая обертка.

Пусть у нас есть Flow, который отправляет строки:

```

1 | val flowStrings = flow {
2 |     emit("abc")
3 |     emit("def")
4 |     emit("ghi")
5 | }

```

Предположим что нам надо эти строки привести к верхнему регистру (закапслчить) и вывести их в лог.

Очевидно, мы сделаем так:

```

1 | flowStrings.collect {
2 |     log(it.toUpperCase())
3 | }

```

Запускаем Flow и для каждой строки выполняем toUpperCase.

Но хотелось бы получить Flow, который будет сам выполнять преобразование к верхнему регистру. А нам, как получателю, останется только вывести в лог. Конечно, для этого можно было бы использовать оператор map, но мы в учебных целях напишем свой оператор.

Итак, у нас есть один Flow - flowStrings. Нам надо для него сделать Flow-обертку, который будет получать элементы flowStrings, преобразовывать их и отправлять.

Создаем новый пустой Flow:

```

1 | flow {
2 |
3 | }

```

Он пока пустой и ничего не делает. Давайте добавлять функционал.

Начнем с того, что нам надо получать элементы из flowStrings. Чтобы получить что-то из Flow, надо вызвать collect. Так и делаем:

```

1 | flow {
2 |
3 |     flowStrings.collect {
4 |
5 |     }
6 |
7 | }

```

Внутри нашего Flow запускаем получение данных из flowStrings.

Далее надо добавить преобразование в верхний регистр:

```

1 | flow {
2 |
3 |     flowStrings.collect {
4 |         it.toUpperCase()
5 |     }
6 |
7 | }

```

Для каждого получаемого из элемента из flowStrings применяем метод toUpperCase.

Не хватает еще кое-чего. Когда мы создаем свой Flow, то мы должны создавать и отправлять данные. Создание данных у нас готово, а вот отправки нет. Добавляем emit.

```

1 | flow {
2 |
3 |     flowStrings.collect {
4 |         emit(it.toUpperCase())
5 |     }
6 |
7 | }

```

В итоге мы создали Flow, который запрашивает элементы из другого Flow, преобразует их и отправляет получателю.

Тут важно понимать один момент. Наш созданный Flow не запустит flowStrings.collect, пока сам не будет запущен. Т.е. весь этот код начнет работу только когда кто-то вызовет collect для нашего созданного Flow.

В итоге мы можем создать кучу таких оберток, одну над другой.

```

1 | flowStrings
2 |     .map {
3 |         // ...
4 |     }
5 |     .filter {
6 |         // ...
7 |     }
8 |     // ...

```

Но ни одна из них не начнет работу, пока не будет вызван метод collect для получившегося Flow.

Упакуем наш код в extension-метод toUpperCase и получим свой оператор для Flow<String>:

```

1 fun Flow<String>.toUpperCase(): Flow<String> = flow {
2     collect {
3         emit(it.toUpperCase())
4     }
5 }

```

Теперь мы можем использовать этот оператор, чтобы Flow со строками преобразовать в Flow с теми же строками, но в верхнем регистре:

```

1 flowStrings.toUpperCase().collect {
2     log(it)
3 }

```

В качестве еще одного примера можно рассмотреть такой код:

```

1 flow {
2     emit("start")
3     flowStrings.collect {
4         emit(it)
5     }
6     emit("end")
7 }

```

Создаем Flow-обертку над flowStrings. Эта обертка просто пересылает данные из flowStrings никак их не трансформируя. Но перед этой отправкой и после нее она отправляет строки start и end. Т.е. таким образом можно сделать операторы типа startWith и endWith.

В примере выше, кстати, конструкцию

```

1 flowStrings.collect {
2     emit(it)
3 }

```

можно заменить кодом

```

1 emitAll(flowStrings)

```

Метод emitAll просто переотправит все элементы из flowStrings.

Важный момент. Когда вы используете map, filter или создаете свой оператор, не забывайте, что используемый в них код преобразования данных будет выполнен в suspend функции collect, которая начнет всю работу Flow. Это значит что ваш код не должен быть тяжелым или блокировать поток. Не забывайте, что suspend функция может быть запущена и в main потоке.

Terminal

Примеры Terminal операторов - collect, single, reduce, count, first, toList, toSet, fold. Названия все знакомые и понятные, не будем разбирать каждый отдельно.

Разберем главное отличие от Intermediate операторов. Intermediate операторы берут Flow, добавляют к нему какое-либо преобразование его данных и возвращают новый Flow. Но они не запускают Flow и не получают результаты его работы.

A Terminal операторы запускают Flow так же, как это делает collect. Соответственно, результатом их работы является не Flow, а данные, полученные из Flow и обработанные определенным образом.

В качестве примера рассмотрим оператор count

```

1 val count = flowOf("a", "b", "c").count()

```

Он посчитает и вернет количество элементов Flow. А для этого ему, очевидно, придется запустить Flow и дождаться, пока он закончит свою работу.

Посмотрим исходники:

```

1 public suspend fun <T> Flow<T>.count(): Int {
2     var i = 0
3     collect {
4         ++i
5     }
6
7     return i
8 }

```

Тут уже не создается никакой Flow, как в Intermediate операторах. Потому что мы сейчас делаем не преобразование данных, а хотим получить данные. Вызывается collect и считается количество полученных элементов.

Т.к. Terminal операторам приходится вызывать collect, все они являются suspend функциями и вызываться могут только в корутине (или в другой suspend функции).

Давайте создадим свой Terminal оператор, чтобы понять тему лучше.

Пусть у нас есть Flow, который отправляет строки:

```

1 | val flowStrings = flow {
2 |     emit("abc")
3 |     emit("def")
4 |     emit("ghi")
5 | }

```

И мы хотим все эти строки собрать в одну.

Для этого нам надо будет запустить Flow и получить все его элементы, т.е. вызвать collect:

```

1 | flowStrings.collect {
2 |
3 | }

```

А в collect блоке нам надо собирать все получаемые элементы в одну строку. Используем StringBuilder:

```

1 | val sb = StringBuilder()
2 |
3 | flowStrings.collect {
4 |     sb.append(it).append(",")
5 | }
6 |
7 | val result = sb.toString()

```

В result мы получим все элементы Flow, собранные в одну строку.

Для упрощения кода я не стал заморачиваться с удалением последнего разделителя.

Помещаем наш код в extension-метод join:

```

1 | suspend fun Flow<String>.join(): String {
2 |     val sb = StringBuilder()
3 |
4 |     collect {
5 |         sb.append(it).append(",")
6 |     }
7 |
8 |     return sb.toString()
9 | }

```

И готов новый Terminal оператор.

```

1 | val result = flowStrings.join()

```

Прочее

Есть еще несколько операторов, которые я хотел бы упомянуть.

Три простых оператора из названия которых сразу понятен их смысл: onEach, onStart, OnCompletion. Все они - Intermediate. Т.е. не запускают текущий Flow, а создают обертку над ним.

А также есть Intermediate оператор transform, который рекомендуется к использованию для создания своих Intermediate операторов.

На вход он получает данные, а вот вместо выходных данных он должен вызывать emit.

Пример:

```

1 | flowStrings.transform { value ->
2 |     emit(value)
3 |     emit(value)
4 | }

```

Этот transform создаст Flow, который будет получать данные из flowStrings и отправлять их дважды. Т.е. мы создали оператор, который можно было бы назвать double.

Еще один пример:

```

1 | flowStrings.transform { value ->
2 |     value.forEach { emit(it) }
3 | }

```

Этот transform каждую полученную строку разбивает на символы (forEach) и каждый из них отправляет отдельно.

Т.е. из Flow<String> мы получим Flow<Char>.